ASSIGNMENT- 11.1

R. Nikhil kumar

2303A52260

Batch: 44

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty Methods

Code:

```python
class Stack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0
    def size(self):
        return len(self._items)

print("=== Stack Demo ===")
stack = Stack()
print("Push 10 and 20 to stack")
stack.push(10)
stack.push(20)
print(f"Peek (top element): {stack.peek()}")
print(f"Pop (remove top): {stack.pop()}")
print(f"Peek after pop: {stack.peek()}")
print(f"Stack size: {stack.size()}")
```

✓ 0.0s

Output:

```
=== Stack Demo ===
Push 10 and 20 to stack
Peek (top element): 20
Pop (remove top): 20
Peek after pop: 10
Stack size: 1
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Code:

```python
class Queue:
    def __init__(self):
        self._items = []

    def enqueue(self, item):
        self._items.append(item)

    def dequeue(self):
        if not self._items:
            raise IndexError("dequeue from empty queue")
        return self._items.pop(0)

    def peek(self):
        if not self._items:
            raise IndexError("peek from empty queue")
        return self._items[0]

    def size(self):
        return len(self._items)

print("=== Queue Demo (FIFO) ===")
queue = Queue()
print("Enqueue 10, 20, 30")
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
print(f"Peek (front element): {queue.peek()}")
print(f"Dequeue (remove front): {queue.dequeue()}")
print(f"Peek after dequeue: {queue.peek()}")
print(f"Queue size: {queue.size()}")
```
✓ 0.0s

Output:

```
=== Queue Demo (FIFO) ===
Enqueue 10, 20, 30
Peek (front element): 10
Dequeue (remove front): 10
Peek after dequeue: 20
Queue size: 2
```

Task 3 – Linked List

```python
class Node:
    """Represents a single node in a singly linked list."""

    def __init__(self, data):
        """Initialize a node with data and a next pointer."""
        self.data = data
        self.next = None


class LinkedList:
    """A singly linked list with insert and display operations."""

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
        """Insert a new node containing data at the end of the list."""
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node

    def display(self):
        """Return a list of all node values from head to tail."""
        values = []
        current = self.head

        while current is not None:
            values.append(current.data)
            current = current.next

        return values

print("=== Linked List Demo ===")
linked_list = LinkedList()
print("Insert 10, 20, 30 to linked list")
linked_list.insert(10)
linked_list.insert(20)
linked_list.insert(30)
print(f"Display all nodes: {linked_list.display()}")
```
✓ 0.0s

Output:

```
=== Linked List Demo ===
Insert 10, 20, 30 to linked list
Display all nodes: [10, 20, 30]
```

Task 4 – Binary Search Tree (BST)

```python
class BSTNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, value):
        if self.root is None:
            self.root = BSTNode(value)
        else:
            self._insert_recursive(self.root, value)

    def _insert_recursive(self, node, value):
        if value < node.value:
            if node.left is None:
                node.left = BSTNode(value)
            else:
                self._insert_recursive(node.left, value)
        else:
            if node.right is None:
                node.right = BSTNode(value)
            else:
                self._insert_recursive(node.right, value)

    def inorder_traversal(self):
        result = []
        self._inorder_recursive(self.root, result)
        return result

    def _inorder_recursive(self, node, result):
        if node is not None:
            self._inorder_recursive(node.left, result)
            result.append(node.value)
            self._inorder_recursive(node.right, result)

print("=== Binary Search Tree Demo ===")
bst = BST()
print("Insert values: 10, 5, 15")
bst.insert(10)
bst.insert(5)
bst.insert(15)
print(f"In-order traversal (sorted): {bst.inorder_traversal()}")
```

✓ 0.0s

Output:

```
=== Binary Search Tree Demo ===
Insert values: 10, 5, 15
In-order traversal (sorted): [5, 10, 15]
```

Task 5 – Hash Table

```python
class HashTable:
    def __init__(self, size=10):
        # Create fixed-size table where each slot stores a chain (list of key-value pairs)
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        # Convert key into a table index
        return hash(key) % self.size

    def insert(self, key, value):
        # Find the bucket using hashed index
        index = self._hash(key)
        bucket = self.table[index]

        # Update value if key already exists in this bucket
        for pair in bucket:
            if pair[0] == key:
                pair[1] = value
                return

        # Otherwise append new key-value pair (chaining for collisions)
        bucket.append([key, value])

    def search(self, key):
        # Find the bucket and scan chain for matching key
        index = self._hash(key)
        bucket = self.table[index]

        for existing_key, value in bucket:
            if existing_key == key:
                return value

        # Return None when key is not found
        return None

    def delete(self, key):
        # Find the bucket and remove matching key-value pair
        index = self._hash(key)
        bucket = self.table[index]

        for i, (existing_key, _) in enumerate(bucket):
            if existing_key == key:
                del bucket[i]
                return True

        # Return False when key does not exist
        return False

print("=== Hash Table Demo (with Chaining) ===")
hash_table = HashTable()
print("Insert: name='Nikhil', age=30")
hash_table.insert("name", "Nikhil")
hash_table.insert("age", 30)
print("Update: name='Anu'")
hash_table.insert("name", "Anu")
print(f"Search 'name': {hash_table.search('name')}")
print(f"Delete 'age': {hash_table.delete('age')}")
print(f"Search 'age' after delete: {hash_table.search('age')}")
```

✓ 0.0s

Output:

```
=== Hash Table Demo (with Chaining) ===
Insert: name='Nikhil', age=30
Update: name='Anu'
Search 'name': Anu
Delete 'age': True
Search 'age' after delete: None
```

<u>Task 6</u> – Graph Representation

```python
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, source, destination):
        self.add_vertex(source)
        self.add_vertex(destination)
        self.adjacency_list[source].append(destination)

    def display_connections(self):
        return self.adjacency_list


graph = Graph()
graph.add_vertex("A")
graph.add_vertex("B")
graph.add_vertex("C")
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "C")
print(graph.display_connections())
```

[3] ✓ 0.0s

Output:

```
{'A': ['B', 'C'], 'B': ['C'], 'C': []}
```

Task 7 – Priority Queue

```python
import heapq


class PriorityQueue:
    def __init__(self):
        self._heap = []

    def enqueue(self, item, priority):
        heapq.heappush(self._heap, (-priority, item))

    def dequeue(self):
        if not self._heap:
            raise IndexError("dequeue from empty priority queue")
        priority, item = heapq.heappop(self._heap)
        return item, -priority

    def display(self):
        return [(item, -priority) for priority, item in sorted(self._heap)]


priority_queue = PriorityQueue()
priority_queue.enqueue("Task A", 2)
priority_queue.enqueue("Task B", 5)
priority_queue.enqueue("Task C", 1)
print(priority_queue.display())
print(priority_queue.dequeue())
print(priority_queue.display())
```

0.0s

Output:

```
[('Task B', 5), ('Task A', 2), ('Task C', 1)]
('Task B', 5)
[('Task A', 2), ('Task C', 1)]
```

Task 8 – Deque

```python
from collections import deque


class DequeDS:
    """A double-ended queue allowing insertion and removal from both ends."""

    def __init__(self):
        """Initialize an empty deque."""
        self._deque = deque()

    def add_front(self, item):
        """Insert an item at the front of the deque."""
        self._deque.appendleft(item)

    def add_rear(self, item):
        """Insert an item at the rear of the deque."""
        self._deque.append(item)

    def remove_front(self):
        """Remove and return the item from the front of the deque."""
        if not self._deque:
            raise IndexError("remove from empty deque")
        return self._deque.popleft()

    def remove_rear(self):
        """Remove and return the item from the rear of the deque."""
        if not self._deque:
            raise IndexError("remove from empty deque")
        return self._deque.pop()

    def display(self):
        """Return a list of all items in the deque."""
        return list(self._deque)


print("=== Deque Demo (Double-Ended Queue) ===")
deque_ds = DequeDS()
print("Add 10 to front, 20 to rear, 5 to front")
deque_ds.add_front(10)
deque_ds.add_rear(20)
deque_ds.add_front(5)
print(f"Deque contents: {deque_ds.display()}")
print(f"Remove from front: {deque_ds.remove_front()}")
print(f"Remove from rear: {deque_ds.remove_rear()}")
print(f"Deque after removals: {deque_ds.display()}")
```

0.0s

Output:

```
=== Deque Demo (Double-Ended Queue) ===
Add 10 to front, 20 to rear, 5 to front
Deque contents: [5, 10, 20]
Remove from front: 5
Remove from rear: 20
Deque after removals: [10]
```

<u>Task 9:</u> Real-Time Application Challenge – Choose the Right Data Structure

```python
class CafeteriaQueue:
    """A Queue-based system to manage cafeteria orders This class follows FIFO (First In First Out) principle, ensuring students are served in the order they arrive."""
    def __init__(self):
        """Initialize an empty order queue."""
        self.orders = []

    def add_order(self, student_name):
        """ Add a student's order to the queue. Parameters: student_name (str): Name of the student placing the order. """
        self.orders.append(student_name)
        print(f"Order added for {student_name}")

    def serve_order(self):
        """
        Serve the first student in the queue.

        Returns:
            str: Name of the student whose order is served.

        Raises:
            IndexError: If no orders are available.
        """
        if self.is_empty():
            raise IndexError("No orders to serve.")
        served = self.orders.pop(0)
        print(f"Order served for {served}")
        return served

    def is_empty(self):
        """
        Check if the queue is empty.

        Returns:
        bool: True if queue is empty, else False.
        """
        return len(self.orders) == 0

    def display_queue(self):
        """Display current orders in queue."""
        print("Current Queue:", self.orders)


# Example Usage
if __name__ == "__main__":
    cafeteria = CafeteriaQueue()

    cafeteria.add_order("Rahul")
    cafeteria.add_order("Sneha")
    cafeteria.add_order("Arjun")

    cafeteria.display_queue()

    cafeteria.serve_order()
    cafeteria.display_queue()
```

✓ 0.0s

Output:

```
Order added for Rahul
Order added for Sneha
Order added for Arjun
Current Queue: ['Rahul', 'Sneha', 'Arjun']
Order served for Rahul
Current Queue: ['Sneha', 'Arjun']
```

<u>Task 10:</u> Smart E-Commerce Platform – Data Structure Challenge

```python
class ProductSearchEngine:
    """
    A simple Product Search Engine using a Hash Table.

    Products are stored using product_id as the key
    for fast lookup, insertion, and deletion.
    """

    def __init__(self):
        """Initialize an empty product catalog."""
        self.products = {}

    def add_product(self, product_id, name, price):
        """
        Add a product to the catalog.

        Parameters:
        product_id (int): Unique ID of the product
        name (str): Product name
        price (float): Product price
        """
        self.products[product_id] = {
            "name": name,
            "price": price
        }
        print(f"Product '{name}' added successfully.")

    def remove_product(self, product_id):
        """
        Remove a product from the catalog.

        Raises:
        KeyError: If product ID does not exist.
        """
        if product_id not in self.products:
            raise KeyError("Product ID not found.")

        removed = self.products.pop(product_id)
        print(f"Product '{removed['name']}' removed successfully.")

    def search_product(self, product_id):
        """
        Search for a product by ID.

        Returns:
        dict: Product details if found.
```

```python
        Raises:
        KeyError: If product ID does not exist.
        """
        if product_id not in self.products:
            raise KeyError("Product ID not found.")

        return self.products[product_id]

    def display_products(self):
        """Display all available products."""
        for pid, details in self.products.items():
            print(f"ID: {pid}, Name: {details['name']}, Price: ₹{details['price']}")


# Example Usage
if __name__ == "__main__":
    system = ProductSearchEngine()

    system.add_product(101, "Laptop", 55000)
    system.add_product(102, "Smartphone", 25000)
    system.add_product(103, "Headphones", 2000)

    print("\nAll Products:")
    system.display_products()

    print("\nSearch Product 102:")
    print(system.search_product(102))

    system.remove_product(103)

    print("\nUpdated Product List:")
    system.display_products()
```

Output:

```
Product 'Laptop' added successfully.
Product 'Smartphone' added successfully.
Product 'Headphones' added successfully.

All Products:
ID: 101, Name: Laptop, Price: ₹55000
ID: 102, Name: Smartphone, Price: ₹25000
ID: 103, Name: Headphones, Price: ₹2000

Search Product 102:
{'name': 'Smartphone', 'price': 25000}
Product 'Headphones' removed successfully.

Updated Product List:
ID: 101, Name: Laptop, Price: ₹55000
ID: 102, Name: Smartphone, Price: ₹25000
```